34. DATA ENGINEERING TEMELLERİ

ETL/ELT • Batch/Streaming • Warehouse/Lake • Orchestration • Data Quality

Modern veri pipeline'ının yüksek seviyeli akışı



Orchestration • Quality • Metadata • Monitoring

Bu modülün amacı
Bir veri bilimcinin üretimde kullandığı verinin nereden geldiğini, nasıl taşındığını, dönüştürüldüğünü, saklandığını ve güvenilir tutulduğunu anlamak. Amaç veri mühendisi olmak değil; data pipeline mimarisini okuyabilen ve doğru veri ihtiyaçlarını tarif edebilen bir veri bilimci olmak.

Kazanım	Modül sonunda
ETL/ELT	Extract-load-transform akışlarını ve trade-off'larını açıklayabilirsin.
Batch/streaming	Freshness ihtiyacına göre doğru processing modelini seçebilirsin.
Storage	Warehouse, lake ve lakehouse farklarını ayırabilirsin.
Modelleme	Fact/dimension, star schema ve grain kavramlarını anlayabilirsin.
Quality	Null, duplicate, freshness, schema ve referential integrity testleri tasarlayabilirsin.
Orchestration	DAG, retry, idempotency ve backfill kavramlarını uygulama düzeyinde anlayabilirsin.

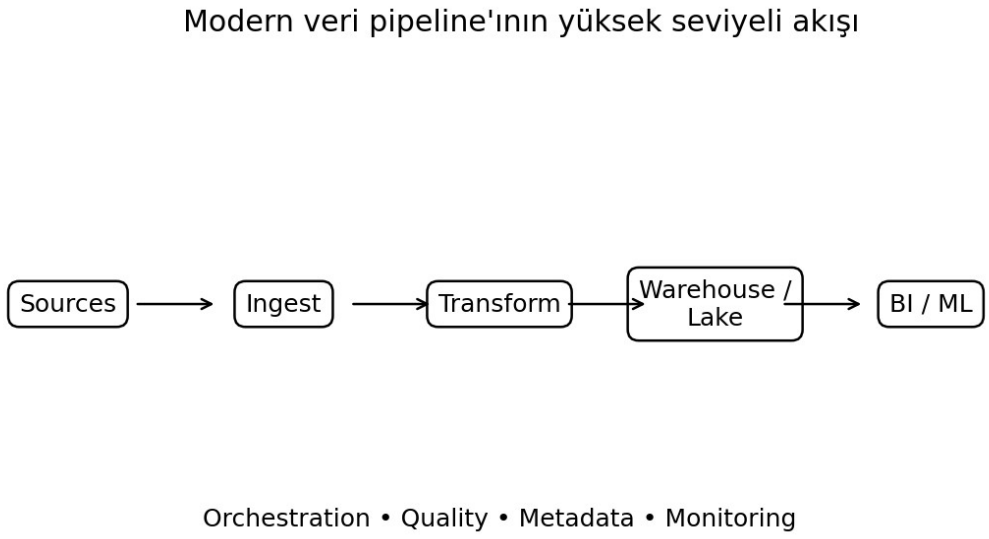
1. Data Engineering nedir?

Data engineering; farklı kaynaklardaki veriyi güvenilir, tekrar üretilebilir, ölçeklenebilir ve tüketilebilir hale getirmek için kurulan sistemler bütünüdür.

Soru	Data Engineering cevabı
Veri nereden geliyor?	DB, API, files, events, SaaS
Nasıl taşınıyor?	Batch ingestion veya streaming
Nerede saklanıyor?	Warehouse, lake, lakehouse
Nasıl dönüştürülüyor?	SQL/Spark/dbt/Python
Ne zaman çalışıyor?	Scheduler/orchestrator
Doğru mu?	Data quality ve observability

Veri bilimci için
Model accuracy'den önce doğru ve zamanında veri gerekir. Pipeline bozuksa model de güvenilmezdir.

2. Modern veri pipeline'ı



- Source systems
- Ingestion
- Raw storage
- Transformations
- Serving layer
- BI / analytics / ML
- Monitoring + metadata + governance

3. Source sistemler

Kaynak	Örnek
Relational DB	PostgreSQL, MySQL
NoSQL	MongoDB, DynamoDB
SaaS	Stripe, Salesforce
Files	CSV, JSON, Parquet
APIs	REST/GraphQL
Events	Kafka-style streams
Logs	Application/server logs

4. ETL ve ELT

Yaklaşım	Akış	Ne zaman?
ETL	Extract -> Transform -> Load	Dönüşüm yüklemeden önce gerekiyorsa
ELT	Extract -> Load -> Transform	Modern warehouse/lakehouse compute güçlü ise

Modern analytics
Bulut warehouse dünyasında ELT yaygındır: raw veri önce merkezi depoya alınır, transformation içeride yapılır.

5. Ingestion

Ingestion, kaynak sistemden veriyi hedef storage'a güvenilir biçimde aktarmaktır.

Pattern	Açıklama
Full load	Tüm tabloyu yeniden yükle
Incremental	Yalnız yeni/değişen kayıtlar
CDC	Change Data Capture
Event ingestion	Event geldikçe taşı

Incremental load
Büyük tabloda her gün full load pahalı ve yavaştır; watermark/id/timestamp ile incremental tercih edilir.

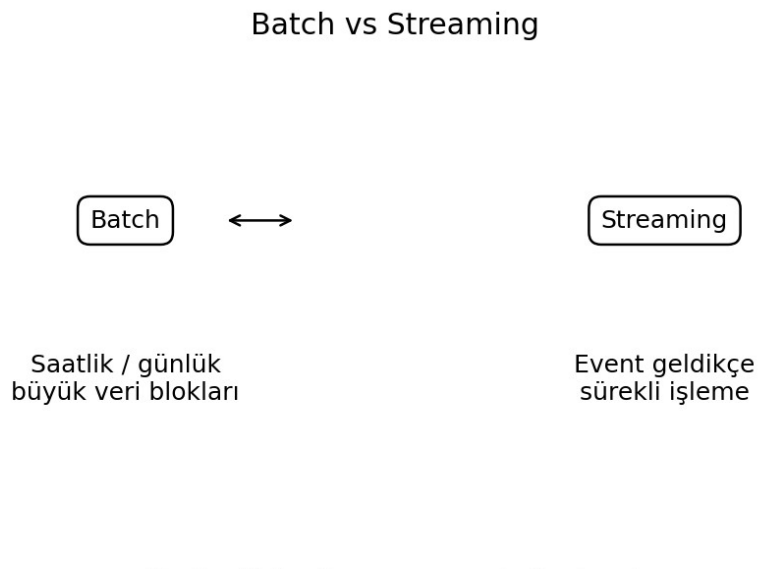
6. Change Data Capture (CDC)

Insert/update/delete değişikliklerini kaynak loglarından veya change stream'den yakalama yaklaşımıdır.

- Daha düşük source load
- Near-real-time updates
- Delete yakalama
- Order ve deduplication ihtiyacı

Veri bilimci açısından
Feature tablosunun güncelliği CDC pipeline'ına bağlı olabilir.

7. Batch vs Streaming



Özellik	Batch	Streaming
Latency	Dakika-saat	Saniye-altı/dakika
Complexity	Düşük	Daha yüksek
Cost	Genelde daha düşük	Sürekli compute
Use case	Daily reporting	Fraud/realtime alerts
Debugging	Daha kolay	State/time karmaşık

Gereksiz streaming kurma
İş ihtiyacı günlük ise real-time mimari yalnız complexity ve maliyet getirir.

8. Event-time vs Processing-time

Zaman	Tanım
Event time	Olayın gerçekten gerçekleştiği zaman
Processing time	Sistemin olayı işlediği zaman

Late data
Event gecikmeli gelirse event-time window'ları watermark ve late-arrival politikası gerektirebilir.

9. Data latency / freshness

Bir dataset'in kaynak gerçeğine ne kadar yakın güncellendiğini ifade eder.

- 5 dakika
- 1 saat
- T+1 günlük

SLA
Freshness iş beklentisi yazılı SLA/SLO olarak tanımlanmalıdır.

10. Data Warehouse

Analytics sorguları için optimize edilmiş merkezi, yapılandırılmış veri deposudur.

Özellik	Warehouse
Schema	Daha kontrollü
Query	SQL analytics
Users	BI/data analysts/data scientists
Storage	Columnar yaygın
Workload	Aggregation/reporting

Örnek teknoloji sınıfı
Snowflake, BigQuery, Redshift ve benzeri analitik warehouse platformları.

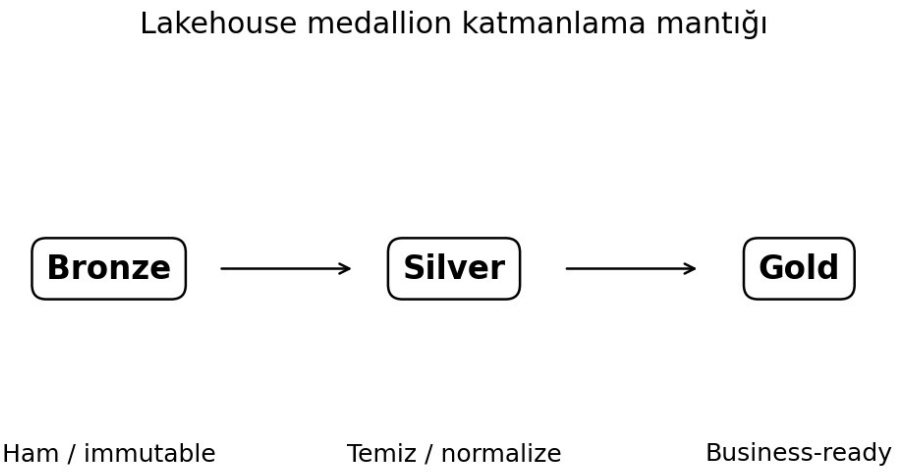
11. Data Lake

Ham ve çeşitli formatlarda büyük veriyi object storage üzerinde saklama yaklaşımıdır.

- CSV
- JSON
- Parquet
- Images/audio
- Logs

Risk
Metadata, schema ve governance yoksa lake kolayca 'data swamp' haline gelebilir.

12. Lakehouse



Lake storage esnekliği ile warehouse transaction/schema/metadata özelliklerini birleştirmeye çalışan mimaridir.

Katman	Amaç
Bronze	Ham, source'a yakın
Silver	Temizlenmiş, normalize
Gold	Business-ready / aggregates

13. File formatları

Format	Artı	Eksi
CSV	Basit ve evrensel	Schema/typing zayıf
JSON	Nested esnek yapı	Verbose
Parquet	Columnar + compressed	Human-readable değil
Avro	Row-oriented + schema	Analytics için Parquet kadar ideal değil

Analytics default
Büyük analitik datasetlerde Parquet çok yaygın ve güçlü bir seçimdir.

14. Row vs Column storage

Storage	Avantaj
Row-oriented	Tek satır transaction read/write
Column-oriented	Belirli kolonları tarayan analytics

Neden Parquet?
Sorgu yalnız 5 kolonu kullanıyorsa tüm satır payload'ını okumak yerine gerekli column chunks taranabilir.

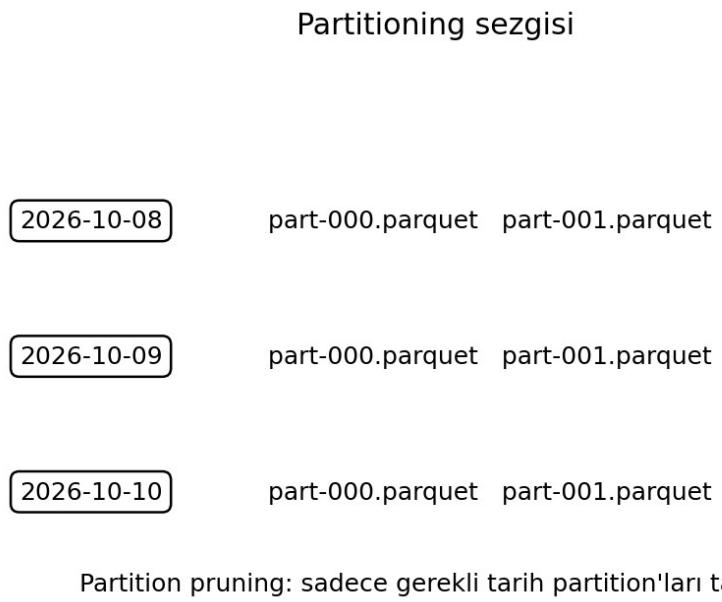
15. Compression

Columnar storage benzer değerleri yan yana tuttuğu için compression genellikle daha etkilidir.

- Snappy
- ZSTD
- GZIP

Trade-off
Daha yüksek compression CPU maliyetini artırabilir ama storage/network maliyetini düşürür.

16. Partitioning



Dataset'i tarih, region veya başka anahtara göre fiziksel parçalar halinde saklama yaklaşımıdır.

Partition pruning
Query date='2026-10-10' ise yalnız ilgili partition okunabilir.

17. Partition anti-patterns

- Aşırı yüksek cardinality ile milyonlarca küçük partition.
- Her customer_id için partition.
- Sorgularda hiç filtrelenmeyen kolonla partition.
- Çok küçük dosyalar.

Small files problem
Object storage'da binlerce küçük dosya metadata/listing ve scheduler overhead yaratabilir.

18. Bucketing / clustering

Partition kadar coarse olmayan ama data locality sağlayan fiziksel düzenleme teknikleridir.

Kullanım
Sık join/filter yapılan yüksek-cardinality kolonlarda bazı sistemlerde faydalıdır.

19. Data modeling - grain

Bir tablodaki her satırın neyi temsil ettiğidir.

Tablo	Grain
orders	Bir sipariş
order_items	Bir sipariş satırı
daily_customer_features	Bir müşteri-bir gün
daily_sales	Bir mağaza-bir gün

En kritik soru
Bu tablonun bir satırı tam olarak nedir? Join hatalarının çoğu grain belirsizliğinden çıkar.

20. Fact ve Dimension

Tür	İçerik
Fact	Ölçülebilir event/transaction
Dimension	Açıklayıcı entity attributes

Örnek
sales fact: revenue, quantity; customer dimension: segment, city, signup_date.

21. Star schema

Merkezde fact table, etrafında dimension tabloların bulunduğu analitik modelleme yaklaşımıdır.

- Basit BI sorguları
- Net grain
- Reusable dimensions
- Predictable joins

22. Slowly Changing Dimensions

Tip	Mantık
Type 1	Eski değeri overwrite et
Type 2	Tarihsel versiyonları koru

ML için önemli
Customer segment history gerekiyorsa bugünkü segmenti geçmiş prediction tarihine join etmek leakage yaratabilir.

23. Surrogate key

Business key yerine warehouse tarafından üretilen teknik dimension anahtarı.

Neden?
Source ID değişiklikleri, SCD Type 2 ve çoklu kaynak entegrasyonunu yönetmeyi kolaylaştırır.

24. SQL transformation layer

```
SELECT
  customer_id,
  DATE(order_ts) AS order_date,
  COUNT(*) AS order_count,
  SUM(amount) AS revenue
FROM raw.orders
WHERE order_ts >= CURRENT_DATE - INTERVAL '30 day'
GROUP BY 1, 2;
```

Data engineering SQL
SELECT yazmaktan fazlasıdır: grain, incremental logic, deduplication, idempotency ve testability önemlidir.

25. Incremental model

Tüm history'yi yeniden hesaplamak yerine sadece yeni/değişen partition veya kayıtlar işlenir.

```
WHERE updated_at > :last_successful_watermark
```

Watermark
Son başarılı run'ın işlediği en ileri timestamp/id.

26. Idempotency

Aynı pipeline aynı input ile tekrar çalıştırıldığında sonucu bozmamalı veya duplicate üretmemelidir.

Risk	Idempotent çözüm
Retry duplicate insert	MERGE/upsert
Same file twice	File checkpoint/id
Re-run partition	Overwrite partition
Event duplicate	Event id dedup

Production kuralı
Retry kaçınılmazdır; pipeline retry-safe olmalıdır.

27. Deduplication

```
WITH ranked AS (
  SELECT
    *,
    ROW_NUMBER() OVER (
      PARTITION BY event_id
      ORDER BY ingested_at DESC
    ) AS rn
  FROM events
)
SELECT *
FROM ranked
WHERE rn = 1;
```

Duplicate ne demek?
Business key ve event semantics tanımlanmadan duplicate tespit edilemez.

28. Upsert / MERGE

Var olan kaydı update, yoksa insert etme pattern'idir.

CDC sink
Incremental/CDC pipeline'larında hedef tabloyu güncel tutmak için yaygındır.

29. Orchestration

Birden fazla task'in dependency, schedule, retry ve state yönetimidir.

Kavram	Anlam
DAG	Task dependency grafiği
Schedule	Ne zaman çalışacak?
Retry	Geçici hata sonrası tekrar
Backfill	Geçmiş dönemleri yeniden çalıştırma
SLA	Beklenen tamamlama zamanı
Sensor	External condition bekleme

Araç sınıfı
Airflow/Dagster/Prefect gibi orchestrator'lar bu problemi çözer.

30. DAG düşüncesi

```
extract_orders
  >> clean_orders
  >> build_daily_sales
  >> run_quality_checks
  >> publish_dashboard_table
```

Dependency
Downstream task ancak upstream başarıyla tamamlandığında başlamalı.

31. Retry ve failure handling

- Transient network error -> retry
- Schema mismatch -> fail fast
- Bad data -> quarantine / alert
- Partial output -> atomic publish

Her hatayı retry etme
Deterministic data/schema hatası retry ile çözülmez; gereksiz load üretir.

32. Backfill

Pipeline logic değiştiğinde veya geçmiş veri eksik olduğunda eski tarih partition'larını yeniden üretme sürecidir.

Tasarım gereği
Pipeline tarih parametresiyle çalışabiliyorsa backfill kolaylaşır.

33. Data quality boyutları

Boyut	Test örneği
Completeness	null_rate < %1
Uniqueness	order_id unique
Validity	amount >= 0
Freshness	max(event_time) son 30 dk
Consistency	currency code whitelist
Referential integrity	customer_id dimension'da var

34. Data quality testleri

```
assert df["order_id"].is_unique
assert df["customer_id"].notna().mean() > 0.999
assert (df["amount"] >= 0).all()
```

SQL/dbt mantığı
not_null, unique, accepted_values, relationships gibi declarative testler yaygındır.

35. Schema evolution

Kaynak şeması zaman içinde değişir: yeni kolon, type değişimi, rename, nested yapı değişimi.

Değişim	Risk
Yeni nullable kolon	Genelde backward-compatible
Kolon silme	Downstream kırılabilir
Type change	Parse/runtime hata
Rename	Contract kırılır

Data contract
Producer ve consumer arasında beklenen schema/semantics açıkça tanımlanır.

36. Data contracts

- Column names
- Types
- Nullable
- Semantic definition
- Freshness
- Ownership

Amaç
Kaynak takım değişikliği downstream pipeline'ları sürpriz şekilde kırmasın.

37. Metadata ve lineage

Metadata	Örnek
Technical	schema, partitions, size
Business	revenue tanımı
Operational	last run, freshness
Lineage	Bu tablo hangi kaynaklardan geldi?

Lineage
Bir KPI yanlışsa upstream hangi transformation'a kadar takip edebileceğini gösterir.

38. Data catalog

Dataset'leri keşfetmek, owner/definition/schema/lineage görmek için merkezi metadata katmanı.

Veri bilimci için
'Customer churn table hangisi?' sorusunu Slack'te sormak yerine catalog'dan bulmak ideal.

39. Observability

Pipeline yalnız 'job succeeded' ile izlenmez; data behavior da izlenir.

- Freshness
- Volume
- Schema
- Null rates
- Distribution
- Pipeline duration
- Cost

Silent failure
Pipeline success olabilir ama yanlışlıkla 0 satır üretmiş olabilir. Data observability bu yüzden ayrıdır.

40. Logs, metrics, alerts

Sinyal	Örnek
Log	Task neden fail oldu?
Metric	Rows processed, duration
Alert	Freshness SLA breached
Trace/lineage	Hata hangi upstream'den geldi?

41. Object storage

Cloud data lake'lerin temel depolama katmanıdır.

- Cheap scalable storage
- Files/objects
- Compute-storage separation
- Lifecycle policies

Path tasarımı
Date/source/table partition yapısı downstream sorgu ve maintenance'i etkiler.

42. Compute-storage separation

Storage kalıcı ve ucuz; compute ihtiyaca göre ayrı ölçeklenir.

Avantaj
Aynı veriye BI, Spark ve ML farklı compute cluster'larından erişebilir.

43. Small files problem

Çok sayıda küçük dosya scheduler, metadata ve listing overhead'i yaratır.

- Compaction
- Target file size
- Partition cleanup

Lakehouse maintenance
Compaction ve file optimization operational bir görevdir.

44. ACID table formats

Object storage üzerinde transaction, schema evolution ve versioning özellikleri sağlayan table format yaklaşımı.

- Atomic writes
- Snapshots
- Time travel
- Schema evolution

Lakehouse çekirdeği
Delta/Iceberg/Hudi benzeri table formatlar bu sınıftadır.

45. Time travel

Dataset'in geçmiş snapshot'ına sorgu yapabilme.

ML reproducibility
Modelin training aldığı veri snapshot'ını tekrar kurmak için çok değerlidir.

46. Data privacy ve access control

- Least privilege
- PII masking
- Row/column-level security
- Encryption
- Retention/deletion

ML riski
Training dataset'e gereksiz PII almak hem governance hem security riskidir.

47. Dev / test / prod ayrımı

Environment	Amaç
dev	Geliştirme
test/staging	Integration/validation
prod	Gerçek workload

Aynı tabloya elle dokunma
Production transformation kod ve deployment pipeline üzerinden yönetilmelidir.

48. CI/CD for data

- SQL lint
- Unit tests
- Data tests
- Schema checks
- Deployment
- Backfill plan

Data code da code'dur
Git, pull request ve automated tests data pipeline'larında da geçerlidir.

49. Data engineering ve ML

ML ihtiyacı	Data engineering karşılığı
Training dataset	Historical point-in-time joins
Features	Reusable feature pipelines
Online inference	Low-latency feature serving
Retraining	Scheduled/triggered pipeline
Monitoring	Freshness/drift metadata

Feature pipeline
Notebook'ta üretilen feature production'da da aynı semantics ile üretilebilmelidir.

50. Point-in-time correctness

Training satırında yalnız prediction timestamp'e kadar bilinen bilgilerin kullanılması gerekir.

Data engineering görevi
Historical joins ve slowly-changing dimensions doğru kurulmazsa model future leakage alabilir.

51. Feature store bağlantısı

Feature definition'larını reusable, versioned ve offline/online uyumlu hale getirme yaklaşımıdır.

Katman	Amaç
Offline store	Training / batch scoring
Online store	Low-latency inference
Registry	Feature definitions/ownership
Materialization	Offline -> online sync

52. Gerçek vaka: e-commerce analytics pipeline

Katman	Çözüm
Sources	orders, customers, products
Ingestion	CDC + daily product snapshot
Raw	Bronze object storage
Transform	Silver cleaned orders
Model	Gold daily_sales + customer_360
Quality	order_id unique, amount >= 0
Serve	BI + churn features

53. Gerçek vaka: fraud event pipeline

İhtiyaç	Mimari
<1 dk freshness	Streaming
Events	Transaction topic
State	10m velocity counts
Feature	user_txn_count_10m
Sink	Online feature store
Offline	Same events -> lake for training

Lambda/Kappa tartışması
Aynı business logic'i batch ve stream'de iki kez yazmak consistency riski yaratır; mümkün olduğunca shared semantics önemlidir.

54. Gerçek vaka: daily sales forecasting

Adım	Karar
Raw	POS daily transactions
Transform	Store-product-day aggregate
Late data	T+1 correction window
Partition	sale_date
Backfill	Date parameterized pipeline
Output	Forecast training table

55. Pipeline debugging checklist

1. Source'ta veri var mı?

2. Ingestion watermark doğru mu?

3. Schema değişti mi?

4. Row count beklenen aralıkta mı?

5. Join sonrası satır sayısı patladı mı?

6. Duplicate key oluştu mu?

7. Freshness SLA sağlanıyor mu?

8. Downstream publish atomik mi?

9. Retry idempotent mi?

10. Son başarılı partition hangisi?

56. Sık yapılan hatalar

- Her şeyi CSV ile taşımak.
- Full load ile dev tabloyu her gün baştan okumak.
- Grain tanımlamadan join yapmak.
- Partition'ı yanlış anahtarla tasarlamak.
- Data quality testini manuel kontrol sanmak.
- Pipeline'ı idempotent tasarlamamak.
- Schema change'i production'da ilk kez görmek.
- Backfill senaryosunu düşünmemek.
- Monitoring'i yalnız job status ile sınırlamak.
- Training snapshot/version bilgisini kaydetmemek.

57. Veri bilimcinin bilmesi gereken minimum tool haritası

Kategori	Tool sınıfı
SQL engine	Postgres/warehouse SQL
Object storage	S3/GCS/Blob
Warehouse	BigQuery/Snowflake/Redshift sınıfı
Transform	dbt/SQL/Spark
Orchestration	Airflow/Dagster/Prefect
Streaming	Kafka sınıfı
Compute	Spark
Quality	dbt tests/Great Expectations sınıfı

Ezber değil kavram
Her ürünü bilmek yerine aynı kategorideki araçların çözdüğü problemi anlamak daha değerlidir.

58. Mülakat soruları

Soru	Kısa cevap
ETL vs ELT?	Transform'un load'dan önce veya sonra yapılması.
Batch vs streaming?	Periodik blok işleme vs event geldikçe sürekli işleme.
Warehouse vs lake?	Structured analytics store vs esnek object storage.
Lakehouse?	Lake esnekliği + warehouse/table özellikleri.
Parquet neden?	Columnar, compressed, analytics-friendly.
Partitioning?	Dataset'i fiziksel bölümlere ayırarak pruning sağlamak.
Grain nedir?	Bir satırın temsil ettiği entity/event seviyesi.
Fact vs dimension?	Event metrics vs descriptive attributes.
SCD Type 2?	Dimension history versiyonlarını korur.
Incremental load?	Yalnız yeni/değişen veriyi işlemek.
CDC?	Kaynak değişikliklerini yakalama.
Idempotency?	Tekrar çalıştırınca duplicate/bozuk sonuç üretmemek.
Backfill?	Geçmiş dönemleri yeniden üretmek.
Freshness?	Verinin source'a ne kadar güncel olduğu.
Data lineage?	Dataset'in upstream/downstream köken haritası.

59. Uygulama alıştırmaları

1. CSV orders verisini Parquet'e çevir ve dosya boyutlarını karşılaştır.

2. order_date ile partition edilmiş klasör yapısı tasarla.

3. Full load yerine incremental SQL yaz.

4. Duplicate event_id için ROW_NUMBER ile dedup yap.

5. Fact_orders ve dim_customer için grain tanımla.

6. SCD Type 2 customer plan history tablosu tasarla.

7. Freshness, unique, not_null ve accepted_values testleri yaz.

8. Basit 5-task DAG dependency şeması çıkar.

9. Backfill için date-parameterized pipeline pseudocode yaz.

10. Training dataset için point-in-time correct join tasarla.

60. Mini portföy projesi

Proje: E-Commerce Analytics Data Platform

Dosya / katman	İçerik
data/raw/	Orders/customers/products raw
sql/staging/	Clean + dedup
sql/marts/	Fact/dimension + daily KPIs
orchestration/	Daily DAG
tests/	Quality tests
docs/	Data dictionary + lineage
notebooks/	Consumer analytics demo
README.md	Architecture + SLAs + backfill strategy

CV cümlesi

Designed a reproducible e-commerce analytics pipeline with incremental ingestion, dimensional modeling, partitioned storage, automated data-quality checks, orchestration, lineage documentation and backfill-safe transformations.

61. Final cheat sheet

Kavram	Hatırlanacak fikir
ETL	Transform before load
ELT	Load then transform
Batch	Periodik processing
Streaming	Continuous events
Warehouse	Structured analytics
Lake	Flexible raw storage
Lakehouse	Lake + table semantics
Parquet	Columnar format
Partition	Pruning / locality
Grain	Bir satır neyi temsil ediyor?
Fact	Event/metric
Dimension	Descriptive entity
Incremental	Only new/changed data
CDC	Capture source changes
Idempotency	Retry-safe pipeline
Backfill	Historical reprocessing
Quality	Automated data correctness
Lineage	Data origin/dependency
Freshness	How current is data?
Data contract	Producer-consumer schema promise

Bu modülden sonra
35. Spark bölümünde büyük veri üzerinde distributed DataFrame işlemleri ve PySpark ile ölçekli transformation mantığına geçeceksin.